

Legacy Lens

De un script de SQL Server heredado a documentación y un plan de migración

Nacho Tovar

Trabajo de Fin de Máster — Máster de Desarrollo con IA

El problema

En muchas empresas la lógica de negocio **no está en el código de la aplicación.**

Está enterrada en procedimientos almacenados escritos hace quince años por gente que ya no trabaja allí. Sin documentación.

Cuando alguien plantea modernizar el sistema, el primer muro no es técnico:

Nadie sabe qué hace ese código, ni por dónde empezar sin romper producción.

Hoy se resuelve con un consultor leyendo procedimientos a mano durante semanas.

La idea

Lo que se puede saber con certeza, se **calcula**.

Lo que requiere juicio, se le **pregunta al modelo**.

Por qué esa separación importa

Si a un LLM le pides las dependencias de cincuenta procedimientos, **inventará tablas que no existen**. Es el uso equivocado de la herramienta.

	Análisis estático	Modelo de lenguaje
Produce	Dependencias, métricas, riesgo	Resúmenes, reglas, diseño propuesto
Cómo	Árbol sintáctico real del T-SQL	Prompt con los hechos ya verificados
Garantía	Exacto y reproducible	Interpretación revisable

El parser es `Microsoft.SqlServer.TransactSql.ScriptDom` : el mismo que usa SSMS.

Las dependencias no se infieren. Se leen del AST.

Qué hace

Se le da un `.sql` generado con *Generate Scripts* de SSMS y devuelve:

- **Inventario** de tablas, vistas, funciones, procedimientos y disparadores
- **Grafo de dependencias** real: qué lee, qué escribe, a quién invoca
- **Riesgo de migración** con el desglose completo de por qué
- **Documentación** de cada objeto en lenguaje de negocio
- **Plan de migración por fases** (patrón *strangler fig*)
- Todo exportable como **Markdown**

Demo

usp_CerrarPedido — riesgo 55/100

+15	CURSOR	Lógica fila a fila que hay que replantear
+25	NO_TRANSACTION	Escribe en 4 tablas sin transacción explícita
+15	NO_ERROR_HANDLING	Modifica datos sin TRY/CATCH

Ninguna puntuación es un número suelto. Tiene que poder discutirse con el cliente.

Dos ejemplos, dos formas de estar mal

	Riesgo máximo	Dónde está la deuda
ERP de facturación	55 · alto	cursores, SQL dinámico, transacciones
Almacén y expediciones	80 · crítico	proceso por etapas, lógica repartida, cero garantías

El segundo no tiene **ni un solo cursor**. Con un único ejemplo la herramienta parecía medir siempre lo mismo.

Cuando el sistema admite lo que no sabe

`usp_InformeVentas` construye la consulta concatenando cadenas.

Sus dependencias reales **no se pueden conocer sin ejecutarlo**.

La aplicación lo dice explícitamente, en lugar de fingir que la lista de dependencias está completa.

Un límite del análisis estático que hay que **señalar, no disimular**.

Arquitectura

Domain	entidades y recorridos del grafo. No conoce a nadie.
Application	casos de uso, puertos y behaviours (CQRS con MediatR)
Persistence.EF	} adaptadores: implementan los puertos
Analysis	
Ai	
Web	presentación. Solo ISender.
Mcp	servidor MCP. Solo traduce a MediatR.

Las dependencias apuntan **siempre hacia dentro**. Y **Analysis** no depende de **Ai**.

Si Azure OpenAI falla o no está configurado, el análisis estático se entrega igual y la aplicación sigue siendo útil.

Dos modelos, dos papeles

	Documentar objetos	Plan de migración
Modelo	gpt-4.1-mini	gpt-4o
Llamadas	Una por objeto, en paralelo	Una sola
Contexto	Un procedimiento	El grafo completo

Documentar es trabajo repetitivo de contexto corto. El plan es una única decisión que exige razonar sobre todo el sistema.

Pagar el modelo grande cincuenta veces no mejora el resultado, solo la factura.

Lo medí, y me equivocaba

Construí un arnés de evaluación con un conjunto dorado: las reglas de negocio que sé que están en el código.

Modelo	Cobertura	Inventados	Tokens salida
<code>gpt-4.1-mini</code>	100 %	0	2951
<code>gpt-4o</code>	88 %	0	1832

El modelo económico documenta mejor. `gpt-4o` fue más lacónico y omitió que `usp_CerrarPedido` puede dejar datos inconsistentes.

Cuando los hechos ya vienen verificados en el prompt, documentar no es razonar: es redactar sin dejarse nada.

La alucinación se detecta sola

El parser da el inventario **exacto** del esquema.

Cualquier objeto cualificado que el modelo mencione y no esté en ese inventario es, por definición, inventado.

Sin juicio humano. Sin otro modelo de juez.

Es la decisión de arquitectura del principio, cobrando intereses.

Infraestructura como código

```
Terraform → Azure OpenAI (2 despliegues de modelo)
           Container Registry
           Container Apps
           Azure SQL Database (serverless, solo Entra)
           Log Analytics
```

La aplicación no tiene ningún secreto. Llama a OpenAI, lee el registro y entra en la base

de datos con su **identidad administrada**. El servidor SQL no admite usuario y contraseña.

Un único secreto en todo el proyecto, y está razonado: la clave de la cuenta que guarda el

estado de Terraform, que vive en otra suscripción donde no se pueden asignar roles.

Que no es una demo con truco

33 tests sobre las partes deterministas

- Distingue lecturas de escrituras
- Detecta SQL dinámico en sus dos formas
- No confunde `sp_executesql` con una llamada a procedimiento
- Detecta funciones escalares usadas dentro de expresiones
- La suma de los factores de riesgo siempre cuadra con el total
- El recorrido del grafo aguanta **ciclos**: dos procedimientos que se llaman mutuamente colgarían un recorrido ingenuo

Se puede testear con asserts **porque esa mitad del sistema es determinista.**

Cómo lo construí con IA

Delegué: la exploración de la API del parser, el script de ejemplo, el código repetitivo de la interfaz, la primera versión de los prompts.

Decidí yo: la separación entre lo calculado y lo interpretado, el modelo de dominio, la elección de los dos modelos.

Tuve que corregir a mano dos fallos:

- El analizador perdía las funciones escalares invocadas en expresiones
- Confundía «objetos a los que nadie llama» con «objetos que no llaman a nadie»
→ órdenes de migración **opuestos**

La conclusión práctica

Ninguno de esos dos fallos lo detectó la IA.

Los detectó el volcado de diagnóstico de los tests.

La IA acelera enormemente la parte mecánica.

Los tests siguen siendo lo que separa «compila» de «funciona».

El análisis, dentro de tu agente

Servidor **MCP** sobre las mismas consultas de la capa de aplicación.

Herramienta	Responde a
<code>list_analyses</code>	qué sistemas hay analizados
<code>find_object</code>	qué hace esto, cuánto riesgo tiene, de qué depende
<code>where_used</code>	quién toca esta tabla, y si la lee o la escribe
<code>change_risk</code>	qué se rompe si lo cambio, y qué migrar antes

Deja de ser una herramienta que **consultas** y pasa a ser contexto que tu agente **tiene** mientras migra.

Limitaciones reconocidas

- El SQL dinámico es un límite infranqueable del análisis estático
- Afinidad de sesión: `azurerem` no expone `stickySessions` ; con una réplica no aplica,
pero hay que resolverlo antes de escalar
- Una réplica: el circuito de Blazor Server tiene estado y escalar exige afinidad de sesión
- La documentación generada hay que revisarla: es interpretación fundamentada, no
verdad demostrada

El proyecto no termina aquí

Resuelve un problema que tengo delante en el trabajo. Va a seguir.

Fase		
0	Núcleo del producto	Entregado
1	Evaluación de LLM, DevSecOps, coste visible	Entregado
2	Servidor MCP	Entregado
2	RAG vectorial, PL/SQL y Delphi	Planificado
3	OpenTelemetry, Playwright, PostgreSQL	Planificado
4	Análisis encolado con patrón Outbox	Planificado

Descartado a propósito: microservicios, Kubernetes, *fine-tuning*.

No son pendientes: son decisiones.

Cierre

Legacy Lens **no sustituye** al arquitecto que decide la migración.

Le ahorra las dos primeras semanas de leer procedimientos a mano y le da un mapa con el que empezar a discutir.

Repositorio	<code>github.com/Brainiac1703/legacy-lens</code>
Aplicación	<code>https://ca-legacylens-tfm.bluedesert-728dc156.francecentral.azurecontainerapps.io</code>
Usuario de prueba	<code>demo@legacylens.dev</code> / <code>Demo.1234!</code>